SDR Spectrum Sensing Architecture

Architecture Overview

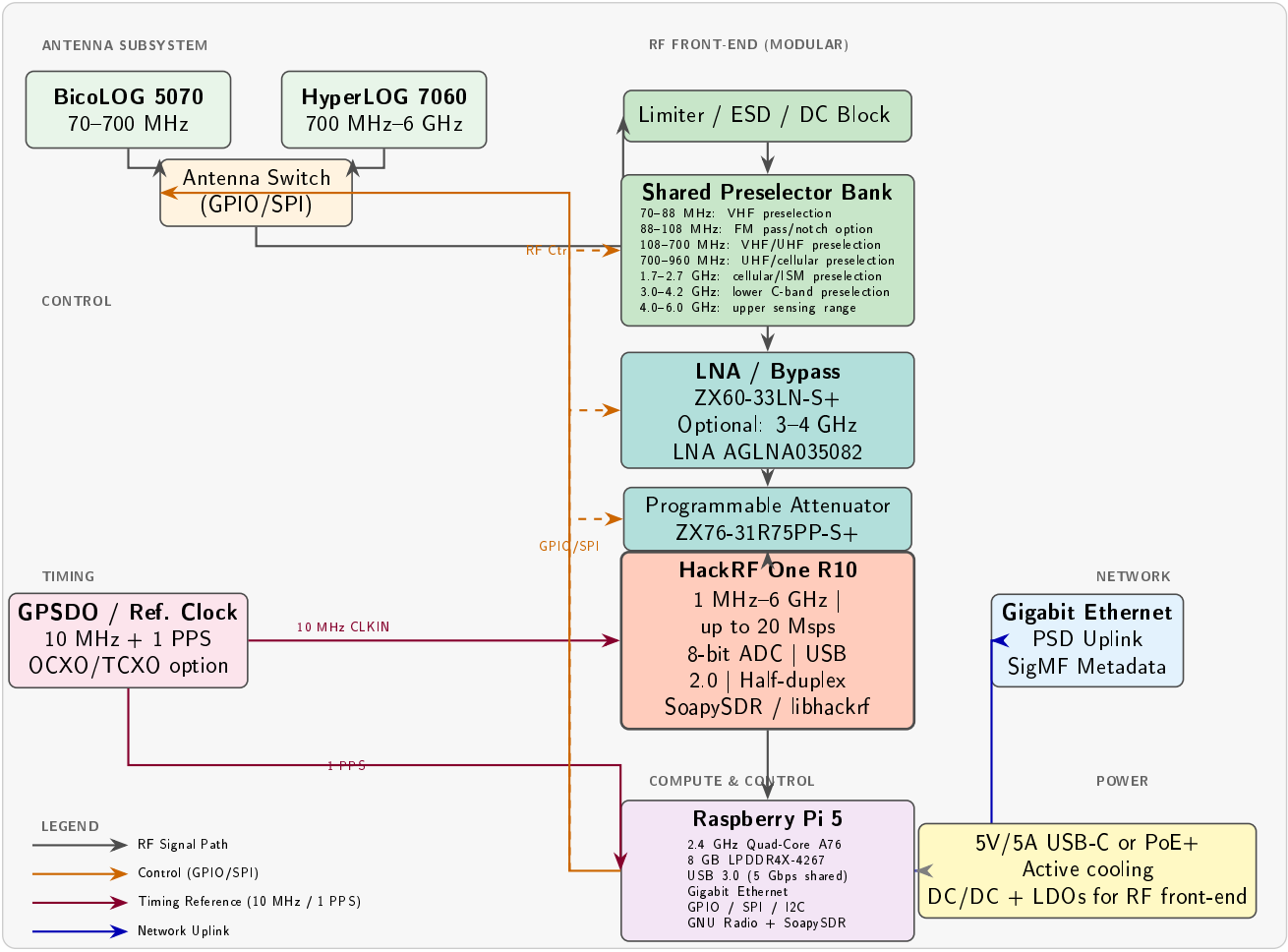

Figure 1: HackRF One R10 + Raspberry Pi 5 70 MHz–6 GHz Spectrum-Sensing Node Architecture

Document Property	Value
Target Frequency Range	70 MHz – 6 GHz (VHF to lower C-band)
SDR Platform	HackRF One R10 (Open Hardware)
Compute Platform	Raspberry Pi 5 (8 GB)
Software Stack	GNU Radio + SoapySDR + SigMF
Estimated Node Cost	~$1,000 USD, depending on RF front-end

1 Component Specifications

SDR Platform: HackRF One R10

Parameter	Specification	Design Implication
Frequency Range	1 MHz–6 GHz	Covers VHF, UHF, L-band, S-band, and lower C-band up to 6 GHz
ADC Resolution	8-bit	Limited instantaneous dynamic range; processing gain possible after averaging
Instantaneous Bandwidth	Up to 20 MHz class	Suitable for channel-level sensing; wideband coverage requires scanning
Sample Rate	2–20 Msps	Sustained operation should be validated on the Pi 5; 10–15 Msps is safer for long captures
Interface	USB 2.0	Compatible with Pi 5 USB 3.0 ports, but limited by HackRF interface throughput
Duplex Mode	Half-duplex	Acceptable for receive-only spectrum sensing
Clock Input	10 MHz CLKIN (SMA)	External reference improves frequency stability for multi-node operation
Trigger I/O	P28 header	Supports hardware-triggered experiments; timing accuracy must be validated
Sensitivity / NF	Configuration-dependent	Requires band-specific calibration; external LNA and preselection are recommended
Approx. SDR Cost	∼$300 USD	Low-cost front-end; full node cost depends on RF modules and timing hardware
Vendor Lock-in	Low	Open hardware with native libhackrf, SoapySDR, and GNU Radio support

Table 1: HackRF One R10 Key Specifications and Design Implications

Critical Design Notes for HackRF One R10

- **8-bit ADC Dynamic Range:** The 8-bit quantizer imposes a limited instantaneous dynamic range. Although averaging, oversampling, and narrowband processing can improve detectability in specific measurement bandwidths, they do not remove the front-end limitation. In RF-dense environments, strong signals may desensitize the receiver or mask weaker adjacent signals. Therefore, the **programmable attenuator**, **LNA bypass**, and **preselector bank**

are essential to reduce ADC saturation and front-end overload.

- **Sensitivity and Noise Figure:** The effective sensitivity of the HackRF One depends strongly on frequency, bandwidth, gain settings, and external RF conditioning. The system noise figure should therefore be measured for each operating band after the selected filter, LNA, attenuator, and gain settings are installed. The external **ZX60-33LN-S+ LNA** can improve weak-signal detectability, but it must be combined with preselection and attenuation to avoid overload in dense RF environments.
- **USB 2.0 Throughput:** The HackRF One supports sample rates up to 20 Msps, but reliable long-duration operation on the Raspberry Pi 5 should be validated experimentally. For continuous monitoring, operating below the maximum rate, for example in the 10–15 Msps range, is a safer design choice. Wider spectral coverage should be obtained through scheduled scanning, frequency hopping, or event-triggered burst capture rather than by assuming continuous full-rate operation.
- **Half-Duplex Constraint:** The half-duplex constraint is not a limitation for receive-only spectrum sensing. If future laboratory signal-injection experiments are required, transmit and receive functions must be time-shared and operated only under controlled and authorized conditions.

Compute Platform: Raspberry Pi 5

Parameter	Specification	Design Implication
CPU	2.4 GHz Quad-Core Arm Cortex-A76	Suitable for FFT-based monitoring, thresholding, metadata generation, and edge reporting
RAM	8 GB LPDDR4X	Adequate for spectrogram buffering, moderate DSP workloads, and local processing queues
USB Ports	2× USB 3.0, shared controller bandwidth	HackRF uses USB 2.0; sustained streaming should still be validated experimentally
Ethernet	Gigabit Ethernet	Supports PSD uplink, metadata reporting, and remote node management
GPIO / SPI / I2C	40-pin header	Controls RF switches, attenuator, LNA bypass, and timing/status signals
Power	5V/5A USB-C recommended	Provides margin for sustained load; full node requires separate RF power budgeting
Storage	MicroSD or NVMe via HAT	NVMe recommended for high-rate IQ logging and long-duration spectrum records
Approx. Cost	Low-cost SBC class	Cost-effective edge platform compared with GPU-enabled embedded computers
GPU / AI	VideoCore VII; no CUDA support	Suitable for CPU-based DSP; heavy ML inference should be offloaded or moved to an AI-capable platform

Table 2: Raspberry Pi 5 Key Specifications and Design Implications

Critical Design Notes for Raspberry Pi 5

- **USB Streaming Reliability:** Although the Raspberry Pi 5 provides USB 3.0 ports, the complete streaming performance depends on the SDR, USB controller behavior, operating-system scheduling, storage load, and GNU Radio processing graph. Since the HackRF One uses USB 2.0 and supports up to 20 Msps, the prototype should validate sustained operation experimentally. For continuous monitoring, operation below the maximum sample rate, for example in the 10–15 Msps range, is a safer starting point.
- **Real-Time DSP Scheduling:** The Raspberry Pi 5 is adequate for FFT-based spectrum monitoring, averaging, thresholding, and metadata generation. For more reliable long-duration operation, the processing pipeline should minimize unnecessary data copies, use bounded queues, and optionally assign CPU affinity to separate DSP, networking, and system tasks.
- **Thermal Management:** Active cooling is recommended for sustained FFT workloads, long-duration streaming, and enclosed operation. The enclosure design should preserve airflow or provide a conductive thermal path so that processor throttling does not reduce DSP throughput during extended measurements.
- **Power Budgeting:** The Raspberry Pi 5 should be powered with a supply that provides sufficient current margin under sustained load. However, the complete sensing node also includes the HackRF, RF switches, LNA, attenuator, GPSDO, storage, and possible DC/DC conversion losses. Therefore, system-level power should be budgeted separately from the Raspberry Pi power requirement.
- **AI and Classification Workloads:** The Raspberry Pi 5 is suitable for lightweight CPU-based classification and edge reporting, but it does not provide CUDA-class acceleration. If AI-based signal classification becomes a central requirement, inference should be offloaded to a central server or migrated to an edge platform with stronger AI acceleration.

2 RF Front-End Module Design

The RF front-end is a mandatory part of the proposed HackRF-based sensing node. Because the HackRF One R10 has limited instantaneous dynamic range, the external RF chain must suppress strong out-of-band signals, protect the receiver input, and provide controlled gain or attenuation before digitization. The front-end is therefore designed as a modular subsystem composed of antennas, RF protection, preselection filters, LNA/bypass stages, and programmable attenuation.

Antenna Subsystem

Antenna	Frequency	Application	Control
BicoLOG 5070	70–700 MHz	VHF/UHF broadcast, PMR, ISM	Antenna switch via GPIO/SPI
HyperLOG 7060	700 MHz–6 GHz	Cellular, WiFi, radar, lower C-band	Antenna switch via GPIO/SPI

Table 3: Antenna configuration for the 70 MHz–6 GHz sensing node

Shared Preselector Bank

The preselector bank suppresses out-of-band RF energy, reduces receiver overload, and lowers the probability of intermodulation products before the HackRF input. This stage is critical because the 8-bit ADC has limited instantaneous dynamic range. Band-specific filtering should therefore be treated as an essential part of the sensing architecture rather than as an optional accessory.

Band	Filter Response	Purpose
70–88 MHz	VHF preselection	Monitor lower VHF while rejecting unnecessary out-of-band energy
88–108 MHz	FM pass/notch option	Pass FM for monitoring or notch it to protect the receiver
108–700 MHz	VHF/UHF preselection	Support VHF/UHF sensing with reduced out-of-band loading
700–960 MHz	Bandpass	LTE 700/800/900 MHz and related UHF services
1.7–2.7 GHz	Bandpass	Cellular, ISM, and microwave communication bands
3.0–4.2 GHz	Bandpass	Lower C-band sensing, radar, and satellite-related signals
4.0–6.0 GHz	Bandpass	Upper sensing range, including WiFi/5G-related bands below 6 GH

Table 4: Preselector filter-bank configuration

LNA and Programmable Attenuator

Component	Specification	Role in Architecture
ZX60-33LN-S+	NF ∼2 dB, gain 15–20 dB, 0.5–4 GHz	Optional LNA for weak-signal sensing; should include bypass control
AGLNA035082 (optional)	NF ∼1.5 dB, 3–4 GHz	Band-specific LNA for improved lower-C-band sensitivity
ZX76-31R75PP-S+	0–31.75 dB, 0.25 dB steps	Programmable attenuation for gain staging and overload prevention

Table 5: LNA and programmable attenuator specifications

Gain-control strategy: Given the limited instantaneous dynamic range of the 8-bit ADC, the receiver should use a coordinated gain-control strategy rather than a simple fixed-gain configuration. The thresholds below should be treated as initial laboratory values and adjusted after calibration.

1. Monitor FFT peak power, broadband RMS level, and clipping indicators over a short update interval, for example every 100 ms.
2. If the peak level approaches the clipping region, for example above −20 dBFS, increase attenuation or disable the LNA.
3. If the received level is low and no strong out-of-band components are present, reduce attenuation or enable the LNA.
4. Use hysteresis, for example 3 dB, to prevent rapid switching between gain states.
5. Log the active antenna, filter path, LNA state, attenuation value, and gain settings as metadata for every capture.

3 Timing and Synchronization

Multi-node scalability benefits from a common frequency reference and a validated time-alignment strategy. In the proposed laboratory prototype, the GPSDO provides a stable 10 MHz reference

for the SDR and a 1 PPS timing signal for node-level timestamping. These signals improve frequency stability and capture repeatability, but system-level timing accuracy must be verified experimentally.

GPSDO Integration

Signal	Source	Destination	Role / Validation Requirement
10 MHz	GPSDO CLKOUT	HackRF CLKIN (SMA)	Frequency reference; verify SDR lock and res
1 PPS	GPSDO PPS OUT	Pi 5 GPIO or timing interface	Timestamp reference; validate GPIO latency

Table 6: Timing-reference distribution for the sensing node

Hardware Trigger Synchronization

The HackRF One R10 P28 header can be used for hardware-triggered capture experiments. This is useful for repeatable laboratory measurements and multi-node capture tests, but sample-level alignment across nodes must be verified experimentally.

- **Trigger output:** A trigger-output signal can be used to coordinate capture events between nodes or external instruments.
- **Trigger input:** A trigger-input signal can be used to initiate or gate acquisition under controlled laboratory conditions.
- **Daisy-chain method:** Nodes may be connected in a trigger chain, for example Node 1 TRIGGER OUT → Node 2 TRIGGER IN → Node 3 TRIGGER IN. This can improve repeatability of capture starts, but inter-node timing accuracy must be measured after accounting for cable delay, trigger propagation, SDR startup behavior, and host-side timestamping.
- **Validation requirement:** For TDOA, phase-sensitive processing, or high-confidence distributed sensing, the system should include a calibration procedure using a common test signal observed by all nodes. The measured timing offsets should be stored as node calibration metadata.

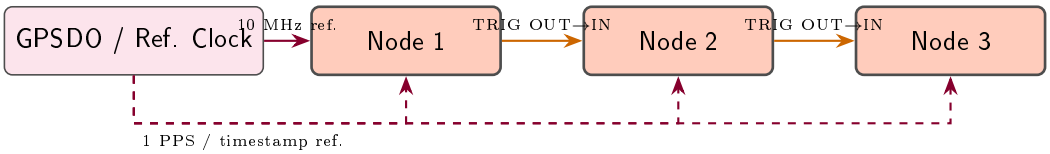

Figure 2: Reference-clock and trigger-assisted multi-node synchronization concept

4 Software Stack and Reduced Vendor Lock-in

The software architecture is designed to reduce vendor lock-in by separating device access, signal processing, metadata handling, and network transport into replaceable layers. The HackRF One R10 is accessed through open driver support, while the processing chain is implemented using portable GNU Radio and Python components. This approach allows the same high-level architecture to be reused with other SDR front-ends, provided that the corresponding driver, gain controls, sample-rate limits, and calibration parameters are adapted.

Layered Software Architecture

Layer	Component	Role in Vendor Portability
SDR Driver	libhackrf / SoapySDR-HackRF	Open driver path for HackRF access
Hardware Abstraction	SoapySDR API	Common SDR interface; reduces hardware-specific flowgraph changes
Signal Processing	GNU Radio 3.10+	Portable DSP framework for streaming, FFT, filtering, and detection
Spectrum Analysis	Python, NumPy, SciPy	Open analysis layer for PSD estimation, logging, and post-processing
Data and Metadata Format	SigMF-compatible metadata	Open capture description for frequency, sample rate, time, gain, and RF path state
Network Transport	ZeroMQ / MQTT / TCP	Open transport options for node telemetry and spectrum-data uplink
Deployment and Services	Docker / Podman / systemd	Portable service deployment across Linux-based hosts

Table 7: Layered software stack for reduced vendor lock-in

GNU Radio Flowgraph Structure

```
[SoapySDR HackRF Source]
|
v
[IQ Conditioning / DC Removal / Gain Metadata]
|
v
[Stream to Vector + Window]
|
+--> [FFT] --> [PSD Estimate] --> [Averaging] --> [PSD Logger]
|
+--> [Threshold Detection] --> [Event Tagger] --> [Burst IQ Recorder]
|
+--> [Metadata Builder] --> [SigMF-Compatible Metadata Sink]
|
+--> [Network Publisher] --> [Central Server / Dashboard]
```

Key Blocks:

- `SoapySDR Source` is used instead of `UHD: USRP Source` for the HackRF-based prototype. This keeps the flowgraph closer to a vendor-portable SDR interface, although device-specific arguments and calibration parameters must still be adapted for each SDR.

- `IQ Conditioning` should include optional DC-offset mitigation, gain-state tracking, clipping indicators, and sample-rate metadata. These steps are important because the HackRF front-end is sensitive to gain staging and strong-signal overload.
- `FFT` processing should use an appropriate analysis window, such as a Hann window, with optional overlap for smoother PSD estimates.
- `PSD Averaging` may use block averaging or exponential smoothing. The averaging time constant should be selected according to the sensing task: shorter averaging for burst detection and longer averaging for occupancy statistics.
- `Threshold Detection` should operate on calibrated or consistently normalized PSD estimates. Detection thresholds should be logged together with the active antenna, filter path, LNA state, attenuation value, center frequency, and sample rate.
- `SigMF-Compatible Metadata` should annotate each capture with frequency, sample rate, timestamp, antenna path, preselector state, gain settings, attenuation value, LNA state, and calibration identifier.
- `Network Publisher` may use ZeroMQ, MQTT, TCP, or another open transport mechanism depending on whether the node is streaming PSD summaries, events, or raw IQ bursts.

5 Power and Thermal Design

Power Budget

The power budget should be treated as a system-level estimate rather than only a Raspberry Pi supply requirement. In addition to the Pi 5 and HackRF One R10, the complete sensing node includes RF-control circuitry, filtering hardware, LNA and attenuator biasing, storage, timing hardware, cooling, and DC/DC conversion losses. For laboratory operation, a 25–30 W design envelope is recommended.

Component	Typical Power	Peak Power
Raspberry Pi 5 with active cooler	7 W	15 W
HackRF One R10	2 W	3 W
LNA / bypass stage	0.5 W	0.8 W
Programmable attenuator	0.1 W	0.2 W
Antenna and filter-switching network	0.1 W	0.5 W
GPSDO / reference-clock module	1 W	2 W
Storage and auxiliary USB devices	0.5 W	2 W
DC/DC and LDO conversion losses	1 W	3 W
Estimated Total Node Power	**12.2 W**	**26.5 W**

Table 8: Estimated per-node power budget including system-level margin

A 5 V/5 A USB-C supply or a properly rated PoE+ power path is recommended for the complete node. If PoE is used, the power budget should include conversion losses and thermal dissipation in the PoE module.

Thermal Considerations

- **Raspberry Pi 5:** Active cooling is recommended for sustained FFT processing, long-duration streaming, and enclosed operation. The enclosure should preserve airflow or provide a conductive thermal path to reduce the risk of CPU throttling during extended measurements.
- **HackRF One R10:** The HackRF One should be mounted to allow heat spreading and airflow around the PCB or enclosure. Passive cooling may be sufficient for laboratory operation, but temperature should be monitored during long captures and high ambient temperature tests.
- **RF Front-End Components:** LNAs, attenuators, switches, and filter modules should be placed to avoid heat concentration near the SDR input. Thermal drift can affect gain, noise floor, and calibration stability, so gain-state and temperature metadata should be logged when possible.
- **Enclosure:** For laboratory use, a ventilated enclosure is sufficient. For outdoor or field deployment, use a weather-resistant enclosure with a controlled thermal path, protected ventilation, or conduction cooling. The internal temperature should be verified experimentally under expected ambient conditions.
- **Power-Supply Margin:** The supply should be sized above the estimated peak load to account for startup current, GPSDO warm-up, storage activity, RF-switch transitions, and conversion losses.

6 Cost Analysis and Scalability

Per-Node Bill of Materials

The bill of materials below separates the low-cost SDR/compute core from the complete sensing node. The final cost depends strongly on the selected antennas, preselector implementation, timing reference, enclosure, storage, and RF-control hardware. Therefore, the values should be interpreted as planning estimates rather than fixed procurement prices.

Component	Qty	Unit Cost (USD)	Extended (USD)
HackRF One R10	1	300	300
Raspberry Pi 5 (8 GB)	1	80	80
Pi 5 active cooler	1	5	5
BicoLOG 5070 antenna	1	150	150
HyperLOG 7060 antenna	1	200	200
Antenna switch / RF-control hardware	1	25	25
Preselector filters or PCB assembly	1	50	50
ZX60-33LN-S+ LNA or equivalent	1	35	35
ZX76-31R75PP-S+ attenuator or equivalent	1	40	40
GPSDO / reference-clock module	1	75	75
Enclosure, cooling, and mounting hardware	1	40	40
Cables, connectors, storage, and miscellaneous	1	30	30
Core SDR + compute subtotal			**385**
Estimated complete node total			**1,030**

Table 9: Estimated bill of materials for one sensing node

Note: The complete-node cost is dominated by antennas, filtering, timing, enclosure, and RF-control hardware. For dense arrays, lower-cost antennas and simplified RF front-ends may reduce the per-node cost, but with reduced sensitivity, selectivity, or calibration quality.

7 Key Limitations and Mitigations

Limitation	Impact	Mitigation Strategy
8-bit ADC	Limited instantaneous dynamic range; strong signals may mask weaker adjacent signals	Use preselection, programmable attenuation, LNA bypass, and conservative gain staging; accept reduced sensitivity as the main cost trade-off
USB 2.0 interface	Limits sustained sample-rate margin and long-duration full-rate capture	Validate sustained streaming on the Raspberry Pi 5; use 10–15 Msps for continuous monitoring and event-triggered burst capture for higher-rate data
Configuration-dependent sensitivity	Weak-signal detectability depends on frequency, bandwidth, gain state, filtering, and LNA configuration	Characterize sensitivity per band; use external LNA only when appropriate and combine it with preselection to avoid overload
Half-duplex operation	Cannot transmit and receive simultaneously	Not a limitation for receive-only spectrum sensing; laboratory signal-injection tests must be time-shared and controlled
Host-side streaming variability	Potential dropped samples under high CPU, USB, storage, or network load	Use bounded queues, CPU affinity when useful, lightweight GNU Radio flowgraphs, local buffering, and sustained-rate validation tests
No CUDA-class acceleration	Limits heavy real-time ML inference at the edge	Keep edge processing focused on FFT, PSD, thresholding, and metadata; offload heavier classification to a central server or AI-capable node
Coverage above 6 GHz	HackRF One R10 does not directly cover frequencies above 6 GHz	Restrict the prototype scope to 70 MHz–6 GHz, or add an external downconverter / higher-frequency SDR for bands above 6 GHz
Thermal stability	Sustained processing and enclosed operation may cause throttling or calibration drift	Use active cooling, airflow or conduction paths, temperature monitoring, and thermal validation under expected operating conditions
Calibration and uncertainty	Uncalibrated PSD, power, frequency, and timing estimates may not be suitable for compliance-grade measurements	Add band-specific calibration, gain-state characterization, frequency-reference validation, timestamp checks, and uncertainty documentation

Table 10: Key limitations and mitigation strategies for the HackRF One R10 + Raspberry Pi 5 sensing node

8 Deployment Recommendations

Indoor/Lab Deployment

- Use the BicoLOG 5070 antenna for 70–700 MHz and the HyperLOG 7060 antenna for 700 MHz–6 GHz, or substitute lower-cost antennas for early bench validation.
- A GPSDO is optional for basic indoor spectral surveys and occupancy monitoring. However, it is recommended when validating frequency accuracy, timing alignment, repeatability, or multi-node operation.
- Use wired Ethernet for reliable PSD uplink, metadata reporting, remote configuration, and central logging.
- Use a standard laboratory enclosure with sufficient ventilation and cable strain relief. For bench testing, an IP20 ABS or metal project box is acceptable, provided that airflow and thermal behavior are verified.

Outdoor/Field Deployment

- Use a weather-resistant enclosure with an appropriate IP rating, protected cable entries, and a controlled thermal path. Passive ventilation should be used only when it does not compromise environmental protection.
- Use a GPSDO or equivalent reference source for frequency stability, timestamping support, and multi-node comparability.
- Use a properly rated power and network solution. PoE+ under IEEE 802.3at or higher-power PoE under IEEE 802.3bt may be used depending on the final node power budget.
- Add RF surge protection, grounding, and lightning protection on outdoor antenna feedlines according to local electrical and safety requirements.
- For off-grid operation, size the solar panel, charge controller, and battery according to the measured node power, required autonomy, expected sunlight, and environmental conditions.

Multi-Node Array Configuration

- Use multiple nodes for distributed sensing, occupancy mapping, and event-triggered comparison across locations. TDOA geolocation generally requires at least three spatially separated nodes, adequate geometry, and validated timing alignment.
- Select node spacing according to the target frequency band, propagation environment, antenna height, terrain, and sensing objective. Larger spacing is appropriate for coarse VHF/UHF coverage, while denser spacing is usually required at higher frequencies or in urban environments.
- Provide each node with a stable frequency reference, such as a GPSDO 10 MHz reference, when frequency comparability or multi-node correlation is required.
- Use trigger-assisted capture only after validating cable delays, trigger propagation, SDR startup behavior, timestamp alignment, and host-side latency. Do not assume sample-synchronous capture without calibration.
- Use a central server to collect PSD summaries, event records, calibration metadata, and selected IQ captures. Store antenna state, filter path, gain settings, attenuation, timestamp source, and calibration identifiers with each capture.

9 Conclusion

The proposed architecture using the **HackRF One R10** and **Raspberry Pi 5** is technically sound as a cost-effective laboratory prototype for spectrum sensing from VHF to the lower portion of C-band (up to 6 GHz). It is well suited to spectral surveys, occupancy monitoring, energy detection, interference screening, signal-presence classification, and event-triggered IQ capture.

The architecture achieves the main design goals as follows:

- **Cost-effective:** The HackRF One R10 + Raspberry Pi 5 core provides a low-cost sensing platform. The complete node cost depends on the selected antennas, RF front-end, timing reference, enclosure, and calibration hardware.
- **Compact:** The SDR and compute platform are small enough for a compact laboratory or field-deployable node, provided that the enclosure also accommodates RF filtering, power regulation, cooling, and cable management.
- **Modular:** The RF front-end is separated from the SDR and compute platform. Antennas, preselectors, LNA/bypass stages, attenuators, timing references, and storage can therefore be upgraded or replaced without redesigning the full system.
- **Scalable:** The design supports a tiered deployment strategy, ranging from dense low-cost sensing nodes to higher-performance anchor nodes. GPSDO references and trigger-assisted capture can support multi-node experiments, but TDOA or sample-level synchronization requires additional calibration and validation.
- **Reduced vendor lock-in:** The system relies on open and portable components, including HackRF hardware, libhackrf, SoapySDR, GNU Radio, Python-based processing, and SigMF-compatible metadata. Device-specific drivers and calibration parameters are still required, but the layered software architecture reduces dependence on a single vendor ecosystem.

The main trade-off is reduced instantaneous dynamic range and sensitivity due to the 8-bit ADC, USB 2.0 throughput constraints, and strong dependence on external RF conditioning. For this reason, the proposed node should be presented as a laboratory-grade spectrum-sensing platform rather than as a calibrated compliance-grade receiver.

For applications requiring weak-signal demodulation, cellular protocol analysis, wide instantaneous bandwidth, or higher dynamic range, the SDR front-end should be upgraded to a higher-performance platform, such as a LimeSDR-class, bladeRF-class, or USRP B200mini-class device. The Raspberry Pi 5 is adequate for FFT-based monitoring, metadata generation, and edge reporting; if AI-based classification becomes a central requirement, inference should be offloaded to a central server or migrated to an edge platform with stronger AI acceleration.